WORLD 4 / GO

Go: The Relay Guild

Train at the Outpost, cross the Signal Tower, and build a relay that can survive a failed job.

The Relay Guild keeps the roads open by carrying messages through a chain of outposts. Its first tests are small. Clean a recruit's badge, read a gate code, and count supplies. Later you will test the dispatch rules, keep workers from sharing state accidentally, and stop waiting when an expedition is canceled.

Start with the Outpost if you are learning Go. It covers values, functions, collections, structs and a JSON boundary. Take the Signal Tower route if you already use those comfortably. It starts at interfaces and goes through error wrapping, testing, goroutines, context and generics. The full route visits both.

Save each example in a fresh main.go and run go run main.go with Go 1.25 or newer. Use gofmt -w main.go before sharing code. At the test lesson, copy the test fragment into a separate clamp_test.go file. The GM provides challenge prompts, hints, and guided review. Run Go to compile and test your drafts.

Start with the beginner route if these ideas are new to you. In the game, choose a hero to save your checkpoint answers, code drafts, and quest log. The attached field kit contains the worked examples, starter files, and a quest journal.

Explore the world map · Begin Level 01

World map

The Outpost

Learn Go by turning recruit and supply records into reliable commands. Start with values and finish at a JSON boundary.

The Signal Tower

Build the relay services once you can read Go functions and collections. Test contracts, coordinate workers, and reason about cancellation and ownership.

Full Relay Expedition

Follow every guild mission in order, from making a first badge to dispatching concurrent work and preserving every result.

The Outpost

01 / A Badge for the Road
02 / The Counter's Ledger
03 / The Watch at Dusk
04 / The Shared Supply Crate
05 / The Quartermaster's Map
06 / The Hero Registry
07 / The Courier's JSON

The Signal Tower

08 / The Relay Interface
09 / A Gate Error With a Trail
10 / The Trial Table
11 / Signals on the Wire
12 / The Recall Signal
13 / One Pattern, Many Types
14 / Capstone - Dispatch the Relay

LEVEL 01 / FIELD MANUAL

A Badge for the Road

Compile a first Go program and use declarations, zero values and string operations.

A runnable Go command belongs to package main and starts at func main. An import makes the exported names of another package available, as in fmt.Println. Go reports unused imports and unused local variables in ordinary local declarations. Keep the command small enough that the role of each name is clear.

Inside a function, := declares at least one new non-blank variable and infers types from the right-hand side. Use = to assign to an existing variable. A declaration such as var visits int gets the zero value 0. Booleans start at false and strings at the empty string.

A Go string is an immutable sequence of bytes. TrimSpace and ToUpper return strings, so save the result or pass it on. len reports bytes. A range loop over a string yields byte indices and decoded runes. A rune is a Unicode code point, and a visible character can contain several code points.

Worked example / Level 01

Listing 1.1 Go example

```
package main

import (
    "fmt"
    "strings"
)

func main() {
    raw := "  Ada  "
    name := strings.ToUpper(strings.TrimSpace(raw))
    var visits int
    visits++
    fmt.Println(name, visits) // ADA 1
}
```

Remember. Use a fresh binding for normalized data and keep byte counts separate from rune counts.

Try the encounter

LEVEL 01 / FORGE

The encounter

Checkpoint

What value does var visits int give visits before any assignment?

A. 1

B. nil

C. 0

D. An unspecified value

Code challenge

Implement badge(raw string) string. Trim leading and trailing whitespace, then uppercase the result. badge(" Jo \t") should return "JO". A whitespace-only input returns the empty string.

Level 1 starter

```
package main

func badge(raw string) string {
    // Trim the recruit name, then uppercase it.
    return raw
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 01 / EXPLAIN

Debrief and side quest

Checkpoint. C. An int has a zero value of 0. Go initializes the variable when it is declared.

Hint one

The strings package contains TrimSpace and ToUpper.

Hint two

Pass the result of strings.TrimSpace(raw) to strings.ToUpper.

Compare your solution

Use strings.TrimSpace before uppercasing, then return the resulting string. Try an all-whitespace name to check the boundary.

Side quest

Print len("é") and the value from a range loop over the same string. Explain the difference.

Continue to Level 02 · World map

LEVEL 02 / FIELD MANUAL

The Counter's Ledger

Return a value and an error, and branch before using a failed result.

A Go function can return more than one value. The conventional signature (T, error) lets callers handle a failure before relying on T. A nil error signals success. The contract describes what the other returned values mean on failure.

strconv.Atoi parses a string as a base-ten int and returns a number plus an error. It accepts an optional sign. Trim outer whitespace if your input contract allows it. A negative value can parse successfully even when the business rule rejects it.

Check the error as close to the call as practical. A short declaration in an if initializer scopes the variables to that if statement and its else branches. Use a declaration before the if when the value must also be used afterward.

Worked example / Level 02

Listing 2.1 Go example

```
package main

import (
    "fmt"
    "strconv"
    "strings"
)

func main() {
    count, err := strconv.Atoi(strings.TrimSpace(" 12 "))
    if err != nil {
        fmt.Println("Unreadable ledger:", err)
        return
    }
    fmt.Println(count + 1) // 13
}
```

Remember. Parsing and domain validation are separate checks.

Try the encounter

LEVEL 02 / FORGE

The encounter

Checkpoint

Why should you check err after strconv.Atoi?

A. An input can fail to parse or exceed int range.

B. Atoi always returns a negative number on failure.

C. An int can store nil.

D. Atoi only reports formatting warnings.

Code challenge

Implement parseCount(raw string) (int, error). Trim outer whitespace, parse a decimal int, and reject negative values. Return zero with a non-nil error for an invalid, overflowing, or negative input. Zero and positive counts succeed.

Level 2 starter

```
package main

func parseCount(raw string) (int, error) {
    // Parse, check the error, then validate the count.
    return 0, nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 02 / EXPLAIN

Debrief and side quest

Checkpoint. A. Atoi reports syntax and range failures through its error result. The number alone cannot represent those outcomes.

Hint one

Use strconv.Atoi on strings.TrimSpace(raw).

Hint two

Return early on a parse error. Use fmt.Errorf or errors.New when a parsed count is negative.

Compare your solution

Try empty text, zero, a negative number, and more digits than int can represent. Each test exercises a different rule.

Side quest

Compare parseCount("+2") with parseCount("2.0") and trace which inputs reach the domain check.

Continue to Level 03 · World map

LEVEL 03 / FIELD MANUAL

The Watch at Dusk

Apply switch, loops, and early returns to a guard that can stop scanning.

Go uses for for counted loops, condition loops and iteration over collections. Range over a slice yields the index and a copy of the element. Use _ to discard an index when the loop only needs the value.

A switch selects the first matching case. Cases finish at the end of their bodies by default. A switch with no expression is equivalent to switching on true, so it can organize a set of predicates.

Returning from inside a loop can express a decisive result. The ready check ends as soon as it sees a blocked task. The return after the loop handles the case where no blocking task was found. In that interpretation an empty list is ready.

Worked example / Level 03

Listing 3.1 Go example

```
package main

import "fmt"

func summarize(states []string) (int, int) {
    ready, blocked := 0, 0
    for _, state := range states {
        switch state {
        case "ready":
            ready++
        case "blocked":
            blocked++
        }
    }
    return ready, blocked
}

func main() {
    fmt.Println(summarize([]string{"ready", "blocked", "ready"})) // 2 1
}
```

Remember. An early return can stop a scan as soon as the outcome is known.

Try the encounter

LEVEL 03 / FORGE

The encounter

Checkpoint

What happens after an ordinary Go switch case executes its body?

A. The next case always executes.

B. The switch finishes unless control was redirected explicitly.

C. The matching case runs again.

D. The default case always executes.

Code challenge

Implement isReady(states []string) bool. Return true when every state is exactly "ready". Return false for any other state. The empty slice counts as ready.

Level 3 starter

```
package main

func isReady(states []string) bool {
    // Find a state that blocks departure.
    return false
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 03 / EXPLAIN

Debrief and side quest

Checkpoint. B. A matching case ends the switch by default. Explicit control flow such as fallthrough or return changes what happens next.

Hint one

Use for _, state := range states.

Hint two

Return false as soon as state differs from "ready". Return true after the loop.

Compare your solution

The post-loop return covers both an empty slice and a slice containing only ready states. Put an invalid state in the middle to check the early exit.

Side quest

Add a separate function that returns the index of the first state that blocks departure. Choose a clear representation for no match.

Continue to Level 04 · World map

LEVEL 04 / FIELD MANUAL

The Shared Supply Crate

Predict when slice operations share storage and make an independent copy on purpose.

An array holds a fixed number of elements and its length is part of its type. A slice describes a segment of an underlying array. Assigning a slice copies the description, which can leave two slices pointing into the same storage.

len is the number of accessible elements. cap is the number of elements the slice can extend to from its starting position. append returns a slice with its updated length. When capacity suffices, append reuses the underlying array. When capacity is too small, it allocates a new one.

Ownership matters when a function keeps or changes a slice. To produce independent storage, allocate a slice with make and copy the elements. That is a shallow copy. Elements containing pointers, maps, or slices may still share objects.

Worked example / Level 04

Listing 4.1 Go example

```
package main

import "fmt"

func main() {
    stock := []string{"rope", "torch"}
    front := stock[:1]
    front = append(front, "map")
    fmt.Println(stock) // [rope map]

    owned := make([]string, len(stock))
    copy(owned, stock)
    owned[0] = "ink"
    fmt.Println(stock[0], owned[0]) // rope ink
}
```

Remember. Use an explicit copy when callers should be able to mutate either slice independently.

Try the encounter

LEVEL 04 / FORGE

The encounter

Checkpoint

What does stock[1] become after appending "map" to stock[:1] in the example?

A. torch

B. The slice index is out of bounds.

C. map

D. The element depends on map iteration order.

Code challenge

Implement withTag(tags []string, tag string) []string. Return a slice containing the original tags followed by tag. The result must have its own storage, including when the input has spare capacity. An empty input produces a one-element result.

Level 4 starter

```
package main

func withTag(tags []string, tag string) []string {
    // Allocate independent storage for every returned tag.
    return append(tags, tag)
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 04 / EXPLAIN

Debrief and side quest

Checkpoint. C. The shorter slice still has capacity for a second element. append reuses that storage and overwrites the slot stock also sees.

Hint one

Allocate a slice of length len(tags)+1.

Hint two

Copy tags into the new slice and assign tag at index len(tags).

Compare your solution

Check the result with an input made using make([]string, 1, 3). Mutate result[0] afterward and verify the input is unaffected.

Side quest

Append to stock[:1:1] and explain what the third index changes. Compare the result with editing front[0] before the append.

Continue to Level 05 · World map

LEVEL 05 / FIELD MANUAL

The Quartermaster's Map

Count values in a map and distinguish missing entries from zero-valued entries.

A map connects keys to values. Key types must be comparable, so strings and integers work directly. A lookup for an absent key yields the zero value of the value type. Use the two-result form v, ok := m[k] when a stored zero must be distinguished from absence.

A nil map permits lookups, len and range. Writing a new entry requires an initialized map, created with make or a map literal. Counting with counts[key]++ works on an initialized map because a missing count reads as zero.

Map iteration order is unspecified. To present stable output, gather the keys into a slice and sort them, then look up each key in that order. Sharing a map with concurrent writers calls for synchronization or a single owner.

Worked example / Level 05

Listing 5.1 Go example

```
package main

import (
    "fmt"
    "sort"
)

func main() {
    counts := map[string]int{}
    for _, crate := range []string{"ink", "rope", "ink"} {
        counts[crate]++
    }
    keys := make([]string, 0, len(counts))
    for key := range counts {
        keys = append(keys, key)
    }
    sort.Strings(keys)
    for _, key := range keys {
        fmt.Println(key, counts[key]) // ink 2, then rope 1
    }
}
```

Remember. Use an initialized map for writes and a sorted key slice for deterministic iteration.

Try the encounter

LEVEL 05 / FORGE

The encounter

Checkpoint

What is ok in n, ok := counts["missing"] when the key is absent?

A. true because n exists

B. false

C. nil

D. The value of n

Code challenge

Implement tally(items []string) map[string]int. Return a non-nil map of exact string counts. Ignore empty strings. Preserve case and whitespace in every other key. A nil input gives an empty map.

Level 5 starter

```
package main

func tally(items []string) map[string]int {
    // Count nonempty keys.
    return map[string]int{}
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 05 / EXPLAIN

Debrief and side quest

Checkpoint. B. The lookup returns the element zero value and false when the key is absent.

Hint one

Initialize a result map and range over items.

Hint two

Skip the empty string with continue. Update other keys with counts[item]++.

Compare your solution

Verify that "rope" and "Rope" occupy separate entries. Use the map's keys and values in tests so iteration order is irrelevant.

Side quest

Add a display helper that lists counts in descending order with alphabetic tie-breaking.

Continue to Level 06 · World map

LEVEL 06 / FIELD MANUAL

The Hero Registry

Model a hero with a struct and choose value or pointer receivers deliberately.

A struct groups named fields into a value. A keyed literal such as Hero{Name: "Mira"} describes which fields are set and leaves other fields at their zero values. Assigning a struct copies its fields. A field that refers to shared storage, such as a slice, still shares it after that copy.

A method has a receiver before its name. A value receiver receives a copy of the receiver value. A pointer receiver can change the original struct through that pointer. Go can automatically take the address of an addressable variable for a pointer-receiver method call.

A constructor is an ordinary function. NewHero can normalize names and reject invalid input before a record is accepted. When an invariant matters across package boundaries, keeping fields unexported can prevent direct writes by callers.

Worked example / Level 06

Listing 6.1 Go example

```
package main

import "fmt"

type Hero struct {
    Name string
    XP   int
}

func (h *Hero) Earn(amount int) {
    h.XP += amount
}

func (h Hero) Label() string {
    return fmt.Sprintf("%s (%d XP)", h.Name, h.XP)
}

func main() {
    h := Hero{Name: "Mira"}
    h.Earn(5)
    fmt.Println(h.Label()) // Mira (5 XP)
}
```

Remember. Use a pointer receiver for a method that updates the caller's struct fields.

Try the encounter

LEVEL 06 / FORGE

The encounter

Checkpoint

Which receiver lets Earn update h.XP on the original hero?

A. (h Hero)

B. (h *Hero)

C. (h string)

D. (h int)

Code challenge

Implement NewHero(raw string) (Hero, error). Trim the name, reject it if empty, and return Hero{Name: cleaned, XP: 0} on success. Return the zero Hero and a non-nil error on failure.

Level 6 starter

```
package main

type Hero struct {
    Name string
    XP   int
}

func NewHero(raw string) (Hero, error) {
    // Normalize and validate the name before constructing a hero.
    return Hero{Name: raw}, nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 06 / EXPLAIN

Debrief and side quest

Checkpoint. B. A pointer to the original Hero allows its field to be updated. A value receiver would copy the int field.

Hint one

Use strings.TrimSpace to normalize the name.

Hint two

Return Hero{} with an error for an empty name. Otherwise the keyed literal can omit XP because its zero value is 0.

Compare your solution

Try a name with internal spaces. Trimming preserves the middle of a name while removing accidental padding.

Side quest

Copy a Hero with a slice of tags. Change a tag through the copy and trace why the original can see it.

Continue to Level 07 · World map

LEVEL 07 / FIELD MANUAL

The Courier's JSON

Decode external data into a Go struct and validate the resulting values.

encoding/json.Unmarshal decodes a JSON value into storage you provide. Pass the address of your target and check the returned error before using it. For a struct, exported fields are eligible for decoding and json tags can choose external names.

Successful decoding checks the JSON shape and conversion rules that the decoder enforces. It does not check application requirements such as a nonempty name or nonnegative XP. A missing field on a fresh struct has its Go zero value. Use a pointer field or explicit presence check when a missing field must differ from a provided zero.

Unmarshal ignores unknown object fields by default. A json.Decoder can call DisallowUnknownFields to enforce a stricter object format. For a single-message boundary, also make sure the decoder has no further JSON value. Choose strictness intentionally when senders and readers evolve separately.

Worked example / Level 07

Listing 7.1 Go example

```
package main

import (
    "encoding/json"
    "fmt"
)

type Hero struct {
    Name string `json:"name"`
    XP   int    `json:"xp"`
}

func main() {
    var h Hero
    err := json.Unmarshal([]byte(`{"name":"Mira","xp":5}`), &h)
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Println(h.Name, h.XP) // Mira 5
}
```

Remember. Decode, check for a conversion error, then enforce your domain rules.

Try the encounter

LEVEL 07 / FORGE

The encounter

Checkpoint

Can json.Unmarshal succeed when a required application field is missing?

A. Yes, because a struct field can keep its zero value.

B. Yes, because every missing field becomes nil.

C. No, because Go requires all struct fields in JSON.

D. Only when the field is unexported.

Code challenge

Implement decodeHero(raw string) (Hero, error). Decode one JSON object, trim the name, and reject blank names or negative XP. Missing XP is accepted as zero. Return Hero{} and a non-nil error on failure. Unknown fields are allowed.

Level 7 starter

```
package main

type Hero struct {
    Name string `json:"name"`
    XP   int    `json:"xp"`
}

func decodeHero(raw string) (Hero, error) {
    // Decode into a local Hero, then validate its fields.
    return Hero{}, nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 07 / EXPLAIN

Debrief and side quest

Checkpoint. A. A missing field stays unchanged. In a fresh Hero, an omitted name is empty and must be rejected by your own validation.

Hint one

Declare var h Hero and pass &h to json.Unmarshal.

Hint two

Trim h.Name and reject an empty name or an XP value below zero after decoding succeeds.

Compare your solution

Use malformed JSON, valid JSON with the wrong XP type, and valid JSON with negative XP to exercise separate failures.

Side quest

Change XP to *int and compare an omitted xp field, an explicit null, and a numeric zero.

Continue to Level 08 · World map

LEVEL 08 / FIELD MANUAL

The Relay Interface

Accept the behavior a function needs and reason about method sets and nil.

An interface describes behavior. A concrete type satisfies an ordinary interface when its method set includes the required methods with matching signatures. A function that accepts io.Reader can read a string in a test and a file in a command while keeping the same implementation.

A value of a defined type T has the methods declared with receiver T. A value of type *T has the methods declared with receiver T or* T. The compiler can automatically take the address for some direct method calls, but a T stored in an interface still needs its own method set to satisfy that interface.

An interface value carries a dynamic type and a dynamic value. Storing a nil *T in an interface retains the dynamic type, so comparing that interface to nil gives false. This matters for error returns. Return a literal nil on the success path when the function returns error.

Worked example / Level 08

Listing 8.1 Go example

```
package main

import (
    "fmt"
    "io"
    "strings"
)

func ReadName(r io.Reader) (string, error) {
    bytes, err := io.ReadAll(r)
    if err != nil {
        return "", err
    }
    return strings.TrimSpace(string(bytes)), nil
}

func main() {
    name, err := ReadName(strings.NewReader("  Mira  "))
    fmt.Println(name, err) // Mira <nil>

    var p *strings.Reader
    var r io.Reader = p
    fmt.Println(p == nil, r == nil) // true false
}
```

Remember. Accept a narrow interface and distinguish a nil interface from one holding a nil pointer.

Try the encounter

LEVEL 08 / FORGE

The encounter

Checkpoint

Why is r == nil false after assigning a nil *strings.Reader to r?

A. io.Reader cannot hold a pointer.

B. r holds the dynamic type *strings.Reader.

C. Assigning p allocates a reader.

D. All interfaces compare unequal to nil.

Code challenge

Implement ReadName(r io.Reader) (string, error) for a non-nil reader with bounded input. Read to EOF, trim outer whitespace, and reject an empty result. Return an empty string and a non-nil error if the read fails or the name is blank.

Level 8 starter

```
package main

import "io"

func ReadName(r io.Reader) (string, error) {
    // Read from r, then validate the trimmed name.
    return "", nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 08 / EXPLAIN

Debrief and side quest

Checkpoint. B. The dynamic type is present even though the dynamic pointer value is nil.

Hint one

Use io.ReadAll, then check its error before converting the bytes to a string.

Hint two

After trimming, return errors.New for an empty name. Test success with strings.NewReader.

Compare your solution

Use a reader that returns an error to check propagation separately from blank input. io.ReadAll is suitable here because the caller supplies bounded input.

Side quest

Define a reader whose Read method has a pointer receiver. Write a compile-time assertion using var _ io.Reader = (*YourReader)(nil).

Continue to Level 09 · World map

LEVEL 09 / FIELD MANUAL

A Gate Error With a Trail

Wrap an error with context while preserving the failure a caller can inspect.

Errors are values. A sentinel such as ErrSealed identifies a failure that callers may handle in a distinct way. Give that sentinel a stable identity by constructing it once. Creating another error with the same message does not give it the same identity.

Use fmt.Errorf with %w when you want to attach context and let callers inspect the underlying error. errors.Is looks for a matching error through wrappers and can use a custom Is method. errors.As finds a compatible error type when the caller needs its fields.

Wrapping an error is part of the caller-facing contract when callers branch on it. Decide which details to expose. The gate example exposes ErrSealed, so a caller can request a key without parsing the human-readable message.

Worked example / Level 09

Listing 9.1 Go example

```
package main

import (
    "errors"
    "fmt"
)

var ErrSealed = errors.New("gate is sealed")

func openGate(name string, hasKey bool) error {
    if hasKey {
        return nil
    }
    return fmt.Errorf("open %s: %w", name, ErrSealed)
}

func main() {
    err := openGate("north", false)
    fmt.Println(err)
    fmt.Println(errors.Is(err, ErrSealed)) // true
}
```

Remember. Wrap with %w when callers need to recognize the cause with errors.Is.

Try the encounter

LEVEL 09 / FORGE

The encounter

Checkpoint

How should a caller identify ErrSealed under the wrapper?

A. err == ErrSealed

B. err.Error() == ErrSealed.Error()

C. errors.Is(err, ErrSealed)

D. errors.New(err.Error()) == ErrSealed

Code challenge

Implement openGate(name string, hasKey bool) error. Return nil if hasKey is true. Otherwise return an error whose message contains name and that matches the provided ErrSealed under errors.Is.

Level 9 starter

```
package main

import "errors"

var ErrSealed = errors.New("gate is sealed")

func openGate(name string, hasKey bool) error {
    // Include the gate name and preserve the sentinel on failure.
    return nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 09 / EXPLAIN

Debrief and side quest

Checkpoint. C. The wrapper and sentinel are different values. errors.Is can inspect the wrapped cause.

Hint one

Return early with nil when the key is present.

Hint two

Use fmt.Errorf with %s for the gate name and %w for ErrSealed.

Compare your solution

Check the message and errors.Is separately. Replacing %w with %v would preserve the text and lose the wrapped sentinel.

Side quest

Define a GateError struct that stores a gate name, implements Error and Unwrap, and can be inspected with errors.As. With Go 1.26 or newer, try errors.AsType[*GateError] if you used a pointer receiver.

Continue to Level 10 · World map

LEVEL 10 / FIELD MANUAL

The Trial Table

Translate a function contract into named table-driven tests.

The go test command discovers Test functions in files ending in _test.go. Test functions take *testing.T. Table-driven tests group input and expected values in rows, then use t.Run to name the result of each case.

For a clamp function, test a value below the bounds, at each bound, between them, and above them. Equal bounds are valid. Reversed bounds need an error. A table records expectations before you look at the implementation and makes it easy to add a missed case.

The Go program below runs independently. To execute the embedded test, copy the lines inside the final comment to clamp_test.go and run go test main.go clamp_test.go. Add a row with reversed bounds and verify that err is non-nil. Run go test -race when testing code that shares memory across goroutines. It reports races it observes during execution.

Worked example / Level 10

Listing 10.1 Go example

```
package main

import (
    "errors"
    "fmt"
)

func clamp(value, low, high int) (int, error) {
    if low > high {
        return 0, errors.New("invalid bounds")
    }
    return min(max(value, low), high), nil
}

func main() {
    fmt.Println(clamp(14, 0, 10)) // 10 <nil>
}

/*
package main

import "testing"

func TestClamp(t *testing.T) {
    cases := []struct {
        name                    string
        value, low, high, want int
    }{
        {"below", -1, 0, 10, 0},
        {"inside", 4, 0, 10, 4},
        {"above", 14, 0, 10, 10},
    }
    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            got, err := clamp(tc.value, tc.low, tc.high)
            if err != nil || got != tc.want {
                t.Fatalf("got %d, %v; want %d, nil", got, err, tc.want)
            }
        })
    }
}
*/
```

Remember. Test the contract at and around each boundary, including invalid input.

Try the encounter

LEVEL 10 / FORGE

The encounter

Checkpoint

Which pair of bounds should your clamp test expect to produce an error?

A. low = 0, high = 0

B. low = -1, high = 1

C. low = 10, high = 1

D. low = 1, high = 10

Code challenge

Implement clamp(value, low, high int) (int, error). Return 0 and a non-nil error when low exceeds high. Otherwise return the value limited to the inclusive interval with a nil error. Add a table-driven test in clamp_test.go locally.

Level 10 starter

```
package main

func clamp(value, low, high int) (int, error) {
    // Validate the interval and handle both boundaries.
    return value, nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 10 / EXPLAIN

Debrief and side quest

Checkpoint. C. The lower bound exceeds the upper bound. Equal bounds form a valid single-value interval.

Hint one

Test low > high before applying either boundary.

Hint two

For each table row include whether an error is expected. Compare the result only when its value is part of the contract.

Compare your solution

Add cases with equal bounds and negative values. Change the function deliberately to return the input unchanged and check that the table catches it.

Side quest

Write a fuzz test that verifies a valid clamp result is between the bounds and stays unchanged when clamped again.

Continue to Level 11 · World map

LEVEL 11 / FIELD MANUAL

Signals on the Wire

Send work over a channel, drain the values, and close from the producer side.

A go statement starts a goroutine. The goroutine can run independently, so main must wait for any result it needs. An unbuffered channel send waits for a receiver. A buffered send can proceed while space is available.

Range over a channel receives until the channel is closed and all buffered values have been drained. The sender should arrange closure when it can guarantee there will be no further sends. A send on a closed channel panics. A receive from a closed and drained channel gives the element zero value with ok equal to false in the two-result form.

With one producer and one receiver, the producer can defer close and the receiver can range. With many producers, a separate coordinator can wait for them before closing. In Go 1.22 and newer language modes, loop variables declared with := are new each iteration. A predeclared variable reused with = is still shared.

Worked example / Level 11

Listing 11.1 Go example

```
package main

import "fmt"

func squares(nums []int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums {
            out <- n * n
        }
    }()
    return out
}

func main() {
    for n := range squares([]int{2, 3, 4}) {
        fmt.Println(n) // 4, then 9, then 16
    }
}
```

Remember. Designate a channel closer who knows every send has finished.

Try the encounter

LEVEL 11 / FORGE

The encounter

Checkpoint

When does a for range loop over a channel finish?

A. As soon as the buffer is empty.

B. When a received value equals zero.

C. When the channel is closed and drained.

D. When the channel loses its last sender reference.

Code challenge

Implement squareAll(nums []int) []int. Start one goroutine that sends the squares of nums over a channel and closes it. Collect the squares in input order. Return an empty slice for empty input. Assume every square fits in int.

Level 11 starter

```
package main

func squareAll(nums []int) []int {
    // Start a sender, drain its channel, and collect results.
    return []int{}
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 11 / EXPLAIN

Debrief and side quest

Checkpoint. C. An empty open channel can receive more values. Range stops only after closure and delivery of remaining buffered values.

Hint one

Create the channel before starting the goroutine. Close it from the sender when the loop finishes.

Hint two

Range over the channel in squareAll and append each received value.

Compare your solution

Try an empty slice. The sender must still close the channel so the receiver can finish. A consumer that stops early needs a way to tell the producer to exit.

Side quest

Replace the output channel with a buffer of size one and trace which sends can complete before the receiver runs.

Continue to Level 12 · World map

LEVEL 12 / FIELD MANUAL

The Recall Signal

Carry cancellation through a blocking operation and handle channel closure separately.

A context carries a deadline, a cancellation signal, and request-scoped values. Pass it as the first argument to a function that honors cancellation. Create a derived context with context.WithTimeout or context.WithCancel and call its cancel function when the operation is done.

Cancellation is cooperative. A select can unblock when ctx.Done closes. Its cancellation case can return ctx.Err, which reports context.Canceled or context.DeadlineExceeded. The function that receives a context should keep its own blocking operations cancellable where practical.

If several select cases are ready, Go chooses one ready case pseudo-randomly. Checking ctx.Err before select gives already-canceled contexts priority at that check. Afterward, if a signal and cancellation become ready together, either result can be valid. A closed signal channel needs its own two-result receive so an empty string can still be a real message.

Worked example / Level 12

Listing 12.1 Go example

```
package main

import (
    "context"
    "fmt"
    "io"
)

func waitForRune(ctx context.Context, signals <-chan string) (string, error) {
    if err := ctx.Err(); err != nil {
        return "", err
    }
    select {
    case <-ctx.Done():
        return "", ctx.Err()
    case signal, ok := <-signals:
        if !ok {
            return "", io.EOF
        }
        return signal, nil
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    signals := make(chan string, 1)
    signals <- "north"
    fmt.Println(waitForRune(ctx, signals)) // north <nil>
    cancel()
    fmt.Println(waitForRune(ctx, signals)) // context canceled
}
```

Remember. Give cancellation its own select case and use ok to detect a closed input.

Try the encounter

LEVEL 12 / FORGE

The encounter

Checkpoint

What happens if both channel cases are ready when select chooses a case?

A. The first source-order case always runs.

B. The cancellation case always runs.

C. One of the ready cases is selected pseudo-randomly.

D. The program panics.

Code challenge

Implement waitForRune(ctx context.Context, signals <-chan string) (string, error). Return ctx.Err for an already canceled context. Otherwise wait for a signal or ctx.Done. Return io.EOF if the signal channel closes without a value, and allow the empty string as a successful signal.

Level 12 starter

```
package main

import "context"

func waitForRune(ctx context.Context, signals <-chan string) (string, error) {
    // Check cancellation and wait on both channels.
    return "", nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 12 / EXPLAIN

Debrief and side quest

Checkpoint. C. Case order does not give a select branch priority. An initial ctx.Err check only handles cancellation already observed at entry.

Hint one

Check ctx.Err at entry, then select between ctx.Done() and a receive from signals.

Hint two

Use signal, ok := <-signals to distinguish a real empty signal from a closed and drained channel.

Compare your solution

Test with a buffered signal, a closed channel, and a canceled context. A canceled context can be constructed without any sleep.

Side quest

Build a signal producer that selects on ctx.Done as well as its send. Check that stopping the consumer also allows the producer to exit.

Continue to Level 13 · World map

LEVEL 13 / FIELD MANUAL

One Pattern, Many Types

Use type parameters when the same collection algorithm works across element types.

Type parameters let a function state relationships between input and output types. A transform can take a slice of T and a function from T to U, then produce a slice of U. Both types remain visible to the caller and can often be inferred at the call site.

A constraint describes the types a parameter can represent and the operations the implementation can use on it. Use any to store and pass values when you need no arithmetic or comparison. The predeclared comparable constraint lets a function compare values or use them as map keys. An interface key can still panic if it contains a non-comparable dynamic value.

A generic helper should state what it does with the input and output. This transform allocates its own output slice and visits inputs in order. If callback results contain pointers or slices, the output values may still reference existing objects.

Worked example / Level 13

Listing 13.1 Go example

```
package main

import (
    "fmt"
    "strconv"
)

func mapValues[T, U any](items []T, transform func(T) U) []U {
    result := make([]U, len(items))
    for i, item := range items {
        result[i] = transform(item)
    }
    return result
}

func main() {
    labels := mapValues([]int{2, 3, 4}, strconv.Itoa)
    fmt.Printf("%q\n", labels) // ["2" "3" "4"]
}
```

Remember. Use the weakest constraint that supplies the operations the implementation needs.

Try the encounter

LEVEL 13 / FORGE

The encounter

Checkpoint

What lets mapValues use any for both type parameters?

A. Go checks every transform at runtime only.

B. The values are passed to the supplied function and stored without type-specific operations.

C. Go converts every item to a string.

D. The compiler requires that T and U are equal.

Code challenge

Implement mapValues[T, U any](items []T, transform func(T) U) []U. Transform each item once in input order and return a fresh slice. Return a non-nil empty slice for a nil or empty input. Assume transform is non-nil.

Level 13 starter

```
package main

func mapValues[T, U any](items []T, transform func(T) U) []U {
    // Allocate a result and apply transform once for each item.
    return []U{}
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 13 / EXPLAIN

Debrief and side quest

Checkpoint. B. The transform already states the relationship between T and U. The loop does not compare or add values of either type.

Hint one

Allocate make([]U, len(items)) and use range to get each index and item.

Hint two

Assign transform(item) to the matching output index.

Compare your solution

Test an int-to-string transform and a struct-to-bool transform. The same function should compile for both sets of types.

Side quest

Write a generic frequency function and explain why its key parameter needs comparable.

Continue to Level 14 · World map

LEVEL 14 / FIELD MANUAL

Capstone - Dispatch the Relay

Bound active work, collect a result for each job, and keep stable input positions.

The dispatcher uses a channel of indices as its work queue. Use sync.WaitGroup.Go to start and track at most limit workers. Each worker finishes its current job before receiving another index. Send every index, close the channel, and wait for all workers before returning.

Store results in a preallocated slice so completion order does not change their positions. Each job index is sent once, so workers write to distinct elements. Keep the slice structure fixed while they run. WaitGroup synchronization makes completed writes visible to the caller after the wait.

Each result holds the value and error returned by its own job. A failed job does not stop the other jobs. The capstone assumes jobs are non-nil, terminate, and synchronize any state they share with each other. Test the limit by tracking active jobs with atomic operations or a mutex. Use channels to coordinate progress in tests.

Worked example / Level 14

Listing 14.1 Go example

```
package main

import (
    "errors"
    "fmt"
    "sync"
)

type Result struct {
    Value int
    Err   error
}

func dispatch(jobs []func() (int, error), limit int) ([]Result, error) {
    if limit < 1 {
        return nil, errors.New("limit must be positive")
    }
    results := make([]Result, len(jobs))
    indexes := make(chan int)

    var workers sync.WaitGroup
    for range min(limit, len(jobs)) {
        workers.Go(func() {
            for i := range indexes {
                value, err := jobs[i]()
                results[i] = Result{Value: value, Err: err}
            }
        })
    }
    for i := range jobs {
        indexes <- i
    }
    close(indexes)
    workers.Wait()
    return results, nil
}

func main() {
    jobs := []func() (int, error){
        func() (int, error) { return 8, nil },
        func() (int, error) { return 0, errors.New("broken signal") },
        func() (int, error) { return 13, nil },
    }
    results, err := dispatch(jobs, 2)
    fmt.Println(results, err)
    // [{8 <nil>} {0 broken signal} {13 <nil>}] <nil>
}
```

Remember. A fixed worker count bounds active work. Index each result by its original job.

Try the encounter

LEVEL 14 / FORGE

The encounter

Checkpoint

Why does the dispatcher write each result at results[i]?

A. Channel sends always arrive out of order.

B. Workers may finish in a different order from job submission.

C. WaitGroup reverses the result slice.

D. Go requires every slice write to use the channel value.

Code challenge

Implement dispatch(jobs []func() (int, error), limit int) ([]Result, error). Reject limit below 1 even when there are no jobs. Assume all jobs are non-nil and return normally. Run every job exactly once with at most limit active. Preserve both job return values in input order and continue after a job error. Return a non-nil empty slice for no jobs with a valid limit.

Level 14 starter

```
package main

type Result struct {
    Value int
    Err   error
}

func dispatch(jobs []func() (int, error), limit int) ([]Result, error) {
    // Validate the limit, distribute indices, and join all workers.
    return []Result{}, nil
}

func main() {}
```

Read the debrief · Back to the field manual

LEVEL 14 / EXPLAIN

Debrief and side quest

Checkpoint. B. The result position records input order. Worker timing can change without changing the output layout.

Hint one

Allocate the results and start min(limit, len(jobs)) workers. Send indices through a channel.

Hint two

Each worker loops over indices. After sending every index, close the channel and wait before returning results.

Compare your solution

Use blocking test jobs to control completion order and track the peak active count. Include a failing job and verify that a later job still runs.

Side quest

Add context-aware jobs and define a result for each unstarted index. Decide when cancellation should stop the sender and what happens to in-flight jobs.

Revisit the world map · Return to Level 01